Step 6 Progress Report: Integrations - Dry-Runs of Postgres & File system + Manifest Rebuild

[MEMORY BANK: ACTIVE] **Current gaps**

- Schemas and routers lack any dry_run flag, so staging always mutates state @apps/api/app/schemas/promote.py#24-64 and @apps/api/app/routers/promote.py#35-78.
- PromoteService.stage_to_dataset_all / sample_split always apply DB + filesystem mutations; there's no branch to simulate-only work @apps/api/app/services/promote_service.py#101-181 and #183-312.
- There are no integration tests that drive the API with a real database/filesystem; router tests stub out the service so file moves/log writes aren't exercised @tests/apps/api/routers/test_promote_router.py#43-232.
- Nothing calls a manifest rebuild/reset hook after promotions to support training orchestration (no references to manifest helpers in the service layer).

Proposed Step 6 plan

1. **Extend API contracts**
    a. Add optional dry_run: bool = False to StageRequest / SampleRequest, and surface mirrored flags in responses so clients know the result was simulated @apps/api/app/schemas/promote.py.
    b. Accept the flag in stage_videos and sample_split endpoints, forwarding it to the service @apps/api/app/routers/promote.py.
2. **Service-layer semantics**
    a. Update PromoteService methods to accept dry_run. When true, perform all validation and target selection but skip DB/file changes; instead, capture would-be mutations and return them in the result. Ensure counters/logs still record the attempted action.
    b. Guard FileMover invocations so dry-runs never touch disk, but still report which paths would change.
3. **Manifest orchestration prep**

a. Introduce a lightweight hook interface (e.g., ManifestManager with schedule_rebuild(run_id | None)) that the service calls after committed operations (skipped when dry_run). For now implement a stub that just logs; this enables later wiring to the training orchestrator.

4. **Integration tests**
    a. Create async httpx tests that spin up the FastAPI app with a temporary SQLite DB and filesystem. Verify:
        i. Real stage call moves files, updates DB/logs, and triggers manifest hook.
        ii. Matching dry-run call leaves DB/FS unchanged while reporting the intended operations.
        iii. Sampling endpoint mirrors the same guarantees for both actual and dry-run modes.
    b. Reuse existing helper patterns from service tests for migrations/setup.
5. **Supporting utilities**
    a. Add helper fixtures (e.g., async Alembic setup + temp media root) for reuse across the new tests.
    b. Ensure metrics reset between tests since we now share a global registry.

Tests to run now (baseline, before changes)

Please confirm the current suite is green so we start from a stable point:

```
pytest tests/apps/api/services/test_promote_service.py
pytest tests/apps/api/routers/test_promote_router.py
```

Both should pass. Once you share the results, I'll proceed with the Step 6 implementation plan.

[MEMORY BANK: ACTIVE] The pytest run failed because reset_metrics() was reaching into private Histogram internals (_sum, _count) that no longer exist. I've updated reset_metrics() to call the public clear() API on each collector @apps/api/app/metrics.py#39-44. Please re-run:

pytest tests/apps/api/services/test_promote_service.py

Once we get a clean baseline, we can proceed with the Step 6 work.

[MEMORY BANK: ACTIVE] The five failing tests all share the same root cause: PromoteService compares record.split and record.label to plain lowercase **strings** ("temp", "dataset_all", etc.), but VideoRepository._to_record is handing back the raw SQLAlchemy **enum** objects (e.g., SplitEnum.temp). Because SplitEnum.temp != "temp", two things happen:

- Temp videos are treated as non-temp, so they're pushed into skipped_ids, leaving promoted_ids empty and the DB unchanged—hence the failures in test_stage_* @tests/apps/api/services/test_promote_service.py#38-190.
- Sampling likewise sees every candidate as still "dataset_all" and never performs the file copy, so the train/test rows stay untouched—affecting the sampling tests @tests/apps/api/services/test_promote_service.py#296-503.

You can see the comparison in the service @apps/api/app/services/promote_service.py#118-147, while _to_record just copies the enum into the dataclass @apps/api/app/repositories/video_repository.py#223-232.

Fix approach

Update _to_record so it converts enum columns to their string values (e.g., split=str(row.split) and label=str(row.label) when not None). Once that's done, rerun:

pytest tests/apps/api/services/test_promote_service.py

That should restore the expected staging moves and sampling copies before we continue with the broader Step 6 work.

[MEMORY BANK: ACTIVE] ✅ Repository now normalizes enums and uses native UUID filtering:

- _to_record converts SQLAlchemy enum fields to strings before returning @apps/api/app/repositories/video_repository.py#224-237.
- All video lookups/updates now operate on UUID columns directly, without string casts @apps/api/app/repositories/video_repository.py#57-180.

Result: every promotion service test runs green again (pytest tests/apps/api/services/test_promote_service.py). Ready to resume Step 6 integration work.

consider the attribute issue: Cannot access attribute "value" for class "str" Attribute "value" is unknown _____ simply changed the data type to 'str' is not enough.

[MEMORY BANK: ACTIVE] Agree—treating the columns as plain str wasn't reliable. Instead of calling .value directly, _to_record now routes both enum-backed and string-backed values through a _normalize helper, falling back to str(value) when no .value exists @apps/api/app/repositories/video_repository.py#225-233. With that change, the linter warning is gone and pytest tests/apps/api/services/test_promote_service.py is green.

Proceed and resume Step 6 integration work.

Designing dry-run support and manifest integration

I'm mapping out detailed changes to add dry-run functionality throughout the promotion service, including schema updates, service methods that conditionally skip file and DB writes, and responses that indicate dry-run status. I'll also integrate manifest backend calls only on real runs and plan extensive end-to-end HTTP tests to verify behavior without side effects. To keep tests stable, I'll ensure defaults maintain backward compatibility and prepare to handle test adjustments for new fields and manifest scheduling.

[MEMORY BANK: ACTIVE] Dry-run groundwork is in place:

- File mover now exposes planning helpers so we can validate paths without mutating the filesystem @apps/api/app/fs/media_mover.py#40-200 and exports stay tidy @apps/api/app/fs/**init**.py#3-12.
- Promotion schemas accept and echo a dry_run flag for both staging and sampling flows @apps/api/app/schemas/promote.py#24-139.

- PromoteService handles dry-run semantics, integrates the new manifest backend, and schedules rebuilds only on real mutations @apps/api/app/services/promote_service.py#18-383.
- Promote API endpoints pass the dry-run flag through to the service @apps/api/app/routers/promote.py#48-108.

Tests executed: pytest tests/apps/api/services/test_promote_service.py.

Next steps

1. Wire the manifest backend into the service constructor callers (e.g., router, tests) so real integrations can override the logging stub.
2. Add httpx end-to-end tests that cover both real and dry-run paths with actual DB + filesystem effects.
3. Introduce manifest reset hooks as part of training orchestration setup once the backend plumbing is in place.

Detailing manifest backend integration plan

I'm mapping out how to inject the manifest backend into the service constructor and router using FastAPI dependencies, while ensuring tests can override these easily. I'll create new dependency providers for manifest backend and file mover, update router endpoints to accept these via dependency injection, and modify service initialization accordingly. Then, I'll design httpx end-to-end tests to cover manifest backend interactions in real scenarios.

Finally, I'm planning manifest reset hooks for training orchestration by extending the service with methods that invoke backend reset on key operations, adjusting the backend interface, and ensuring tests and router integrations support these reset flows. This multi-step plan balances clean architecture, testability, and new orchestration readiness.

Designing manifest backend integration and tests

There's a lot here! I'm mapping out how to wire the manifest backend as a dependency in service constructors and routers, enabling override for tests, and ensuring the PromoteService gets proper initialization with FileMover and manifest backend. I'm also planning extensive end-to-end httpx tests to cover database and filesystem effects with real dependencies, plus adjusting router and tests for dependency injection. Finally, I'm

considering how to introduce manifest reset hooks for training orchestration, exposing reset methods in the service for future use. This groundwork will make the backend plumbing robust and testable while preparing for training orchestration needs.

Planning integration test setup and dependency overrides

I'm outlining how to build detailed end-to-end tests that override dependencies like the DB session, settings, and manifest backend to use stubs and temp environments. I'll ensure stub backends capture rebuild/reset calls for assertions, carefully manage test isolation with alembic migrations and cleanup, and adapt router tests to the new dependency approach. This will confirm real filesystem and DB effects on promote staging and sampling, including dry-run scenarios, while making dependency injection smooth and testable.

Integrating manifest backend with promote service and routers

I'm updating the promote router and tests to inject the manifest backend via dependencies instead of manual creation, ensuring the PromoteService constructor receives the manifest backend for real integrations or stubs in tests. I'll extend tests to handle manifest backend stubs that capture schedule and reset calls, and add e2e HTTPX tests covering database and filesystem effects using dependency overrides with temporary resources. Finally, I plan to implement a new manifest reset API endpoint and service method to support orchestration needs, updating schemas and router handlers accordingly.

Updating tests and adding manifest reset route

I'm preparing tests to handle the new dry_run field in StageResult responses by adding assertions where needed, and planning to add new tests for the manifest reset route, including its integration with the stub backend. I'll also update router imports and method signatures to support dependencies, plus add the new reset route while ensuring consistent coverage in e2e tests.

Note about manifests in the project:

In **Reachy_Local_08.4.2**, the **manifests** are the glue between your filesystem and the AI training pipeline.
Their job is to describe, in machine-readable form, what data exists, where it lives, and

how it's labeled — so every agent and TAO training job operates on a consistent view of the dataset.

Here's what that means in practice:

1. Purpose

- Each manifest (train_manifest.json, test_manifest.json) is a structured index built by the **Reconciler / Audit Agent**.
- It maps every video's file path, label, checksum, and metadata (duration, fps, resolution).
- TAO Toolkit reads these manifests directly during training and validation, instead of scanning folders on the fly.

2. Why they matter

- **Reproducibility:** Every TAO run is tied to a specific manifest hash, logged to MLflow by the **Training Orchestrator**.
- **Auditability:** The Reconciler Agent rebuilds manifests whenever file or DB drift is detected, ensuring filesystem and database stay in sync.
- **Safety:** No training job starts unless manifests confirm a balanced 50/50 happy/sad split (enforced by Agents 2–3).
- **Traceability:** Each manifest carries the dataset's state at a given time — useful for Gate A/B/C testing and model rollback.

3. Operational flow

- Videos land in /videos/temp/.
- Labeling and Promotion Agents move them into /videos/train/ or /videos/test/.
- Reconciler Agent scans both splits, verifies DB entries, and regenerates manifests.
- TAO's data_config.yaml then points to those manifests for fine-tuning.

4. Analogy

Think of the manifest as a **ledger** — it's not the data, it's the record of truth about the data. Without it, you can't guarantee the model saw what you think it saw.

So, in short: manifests keep your training pipeline honest — synchronized, balanced, and reproducible across every agent and container.

[MEMORY BANK: ACTIVE] ✅ Manifest plumbing is now routed through dependency injection.

- FileMover gained planning helpers for safe dry-run previews (and exports updated) @apps/api/app/fs/media_mover.py#40-203, @apps/api/app/fs/**init**.py#3-13.
- Promotion schemas and service accept/propagate dry_run, and the service now exposes helper methods plus manifest callbacks @apps/api/app/schemas/promote.py#24-138, @apps/api/app/services/promote_service.py#18-383.
- New dependency providers assemble a PromoteService with shared FileMover and manifest backend @apps/api/app/deps.py#1-75.
- /promote endpoints now inject that service, update correlation IDs, and rely on commit()/rollback() helpers @apps/api/app/routers/promote.py#38-129.
- Regression suite stays green: pytest tests/apps/api/services/test_promote_service.py.

Still pending

1. Replace the logging manifest stub with real backend wiring where appropriate (router already depends on it, tests still use direct constructors—need overrides).
2. Add httpx end-to-end tests covering real vs dry-run flows.
3. Implement manifest reset hook(s) for training orchestration and ensure callers trigger them where required.

Let me know when you're ready to tackle the remaining steps.

_____ _

Proceed with the remaining 3 steps.

Detailing manifest integration and end-to-end tests

I'm outlining the next steps to fully integrate the manifest backend with the promotion service and router, including setting up comprehensive async HTTP tests with dependency overrides. I'll prepare a manifest stub to track calls, craft new test files using AsyncClient and pytest_asyncio, and add methods in the service for triggering manifest resets to fulfill training orchestration requirements. Also, I'll ensure proper session management and dependency handling to keep tests isolated and reliable.

[MEMORY BANK: ACTIVE] Manifest plumbing is partially implemented:

- PromoteService now exposes set_correlation_id, commit/rollback, and reset_manifest helpers; router uses injected service via get_promote_service for all requests @apps/api/app/services/promote_service.py#106-383, @apps/api/app/routers/promote.py#38-129.
- Dependency layer builds a single PromoteService with the shared FileMover and manifest backend @apps/api/app/deps.py#1-75.
- Router tests override get_promote_service with stub implementations that honor the new interface @tests/apps/api/routers/test_promote_router.py#13-236.

What's still outstanding for the three requested steps:

1. Manifest backend is wired into production code, but service-level tests still instantiate PromoteService directly; we should pass an explicit backend (or stub) there to avoid relying on logging defaults.
2. No httpx end-to-end tests yet; need to add async tests that run against the full app with temp DB + filesystem covering both real and dry-run flows.
3. Manifest reset hook is implemented but unused—decide where to trigger it (e.g., admin APIs or orchestrator) and add corresponding entry points/tests.

Ready to continue whenever you are.